Z-QcryptGIN Documentation

Dashboard Analytics

Endpoint

GET api.z-tas.com/api/v1/admin/dashboard/analytics

Description

Fetch aggregated analytics data for the dashboard, including user metrics, session activity, and system performance indicators.

Request

Headers

Authorization: Bearer <access_token>

Content-Type: application/json

Query Parameters

None

Successful Response

Status Code

200 OK

Response Body


```
{
 "totalUsers": 12543,
 "activeSessions": 342,
 "successRate": 97.5,
 "failedRate": 2.5,
 "suspiciousActivity": 14
}
```


Field Definitions

Field	Type	Description
totalUsers	integer	Total number of registered users
activeSessions	integer	Number of currently active sessions
successRate	number	Percentage of successful operations (0–100)
failedRate	number	Percentage of failed operations (0–100)
suspiciousActivity	integer	Count of flagged suspicious activities

---

Error Responses

401 Unauthorized

User is not authenticated or token is invalid.

{

"error": "Unauthorized",

"message": "Invalid or missing authentication token"

}

---

403 Forbidden

User does not have permission to access dashboard analytics.

{

"error": "Forbidden",

"message": "You do not have access to this resource"

}

---

500 Internal Server Error

Unexpected server failure.

{

"error": "InternalServerError",

"message": "An unexpected error occurred while fetching analytics"

}

Notes (Important for Backend Design)

- `successRate + failedRate` should ideally ≈ 100%

- Rates should be calculated over a consistent window (e.g., last 24h or last 7 days) — define this clearly in implementation
- `suspiciousActivity` could come from:
    - anomaly detection
    - failed auth spikes
    - unusual IP patterns

Dashboard Authentication Trends

Endpoint

GET api.z-tas.com /api/v1/admin/dashboard/auth-trends

Description

Fetch authentication activity trends over the last 24 hours, structured for time-series visualization (e.g., line charts).

Each data point represents authentication outcomes within a fixed time interval.

Request

Headers

Authorization: Bearer <access_token>

Content-Type: application/json

Query Parameters (Optional but Recommended)

Parameter	Type	Default	Description

interval	string	`hour`	Aggregation interval (`minute`, `hour`)

Successful Response

Status Code

200 OK

Response Body


```
{
 "interval": "hour",
 "data": [
   {
     "timestamp": "2026-03-20T00:00:00Z",
     "successCount": 120,
     "failureCount": 5
   },
   {
     "timestamp": "2026-03-20T01:00:00Z",
     "successCount": 98,
     "failureCount": 8
   },
   {
     "timestamp": "2026-03-20T02:00:00Z",
     "successCount": 150,
```

"failureCount": 3
}
]
}

Field Definitions

Root Object

Field	Type	Description
interval	string	Time grouping used for aggregation
data	array	List of time-series data points

Data Object

Field	Type	Description
timestamp	string (ISO 8601)	Start of the time bucket
successCount	integer	Number of successful authentications
failureCount	integer	Number of failed authentications

Error Responses

400 Bad Request

Invalid query parameter (e.g., unsupported interval)

{

"error": "BadRequest",

"message": "Invalid interval. Supported values are 'minute' or 'hour'"

}

---

401 Unauthorized

{

"error": "Unauthorized",

"message": "Invalid or missing authentication token"

}

---

403 Forbidden

{

"error": "Forbidden",

"message": "You do not have access to this resource"

}

---

500 Internal Server Error

{

"error": "InternalServerError",

"message": "Failed to fetch authentication trends"

}

Design Notes

Think of this like how you'd actually query + structure it:

1. Time Bucketing

- If `interval = hour` → 24 data points
- If `interval = minute` → 1440 data points (careful: heavy)

2. SQL Shape (mentally)

You're basically doing:

GROUP BY time_bucket, status

3. Why separate success/failure instead of rate?

Frontend can:

- plot 2 lines (better visibility)
- compute rate if needed
  → more flexible than sending `%`

4. Edge Case You MUST Handle

Missing time buckets → fill with 0
Otherwise graph breaks visually

Example:

{

"timestamp": "2026-03-20T03:00:00Z",

"successCount": 0,

"failureCount": 0

}

Dashboard Recent Authentication Activity

Endpoint

GET api.z-tas.com/api/v1/admin/dashboard/recent-auth-activity

Description

Fetch a list of recent authentication events for display in a dashboard table.
Includes user, device, authentication method, status, and timestamp.

Request

Headers

Authorization: Bearer <access_token>
Content-Type: application/json

Query Parameters (Recommended)

Parameter	Type	Default	Description
page	integer	1	Page number
limit	integer	10	Number of records per page
status	string	null	Filter (`success`, `failure`)

Successful Response

Status Code

200 OK

Response Body


```
{
 "page": 1,
 "limit": 10,
 "total": 245,
 "data": [
   {
     "userId": "user_12345",
     "device": "N/A",
     "method": "default",
     "status": "success",
     "timestamp": "2026-03-21T10:15:30Z"
   },
   {
     "userId": "user_67890",
     "device": "Windows Chrome",
     "method": "passkey",
     "status": "failure",
     "timestamp": "2026-03-21T10:12:10Z"
   }
 ]
}
```

Field Definitions

Root Object

Field	Type	Description
page	integer	Current page number
limit	integer	Number of records per page
total	integer	Total number of records available
data	array	List of authentication activity records

Data Object

Field	Type	Description
userId	string	Unique identifier of the user
device	string	Device or client used for authentication
method	string	Authentication method (`password`, `passkey`, `otp`, etc.)
status	string	Result of authentication (`success`, `failure`)
timestamp	string (ISO 8601)	Time of the authentication event

Error Responses

400 Bad Request

Invalid query parameters

```
{
 "error": "BadRequest",
 "message": "Invalid pagination or filter parameters"
}
```

---

401 Unauthorized

```
{
 "error": "Unauthorized",
 "message": "Invalid or missing authentication token"
}
```

---

403 Forbidden

```
{
 "error": "Forbidden",
 "message": "You do not have access to this resource"
}
```

---

500 Internal Server Error

```
{
 "error": "InternalServerError",
```

"message": "Failed to fetch recent authentication activity"
}

Design Notes (Important for Implementation)

1. Sorting (Critical)

- Always return **latest first**

ORDER BY timestamp DESC

---

2. Pagination Strategy

Offset-based (simple):

LIMIT ? OFFSET ?

Or cursor-based (better for scale):

- `cursor = lastTimestamp`

---

3. Device Field (Don't Overthink Early)

Start simple:

- User-Agent parsing → `"Windows Chrome"`, `"iPhone Safari"`

Later upgrade:

- structured device info (OS, browser, device type)

---

4. Method Enum (Good Practice)

Keep controlled values:

- `password`
- `passkey`
- `otp`
- `oauth`

---

5. Future-Proofing

You'll likely add:

- `ipAddress`
- `location`
- `riskScore`

So design DTO flexible enough.

User Management – List Users, Search and Filter

Endpoint

GET api.z-tas.com/api/v1/admin/users

Description

Fetch a paginated list of users for the User Management dashboard.
Supports **search**, **filtering**, and **sorting**, designed for both **infinite scrolling** and **page-based navigation**.

Request

Headers

Authorization: Bearer <access_token>
Content-Type: application/json

Query Parameters

Pagination

Parameter	Type	Default	Description
limit	integer	20	Number of records per request
offset	integer	0	Offset for pagination

Search

Parameter	Type	Description
search	string	Search across `name`, `email`, and `phone`

Filters

Parameter	Type	Description
status	string	Filter by user status (`active`, `inactive`, `suspended`)
mfaEnabled	boolean	Filter users by MFA enabled/disabled
lastLoginFrom	string (ISO 8601)	Filter users who logged in after this time
lastLoginTo	string (ISO 8601)	Filter users who logged in before this time

---

Sorting (Optional but Recommended)

Parameter	Type	Default	Description
sortBy	string	`lastLogin`	Field to sort by
order	string	`desc`	`asc` or `desc`

Successful Response

Status Code

200 OK

Response Body


```
{
 "limit": 20,
 "offset": 0,
 "total": 1342,
 "data": [
   {
     "userId": "user_12345",
     "name": "John Doe",
     "email": "john@example.com",
     "phone": "+6591234567",
     "mfaEnabled": true,
     "lastLogin": "2026-03-21T09:45:12Z",
     "status": "active"
   },
   {
     "userId": "user_67890",
     "name": "Jane Smith",
     "email": "jane@example.com",
     "phone": "+6598765432",
     "mfaEnabled": false,
     "lastLogin": "2026-03-20T22:10:05Z",
     "status": "suspended"
   }
 ]
}
```

Field Definitions

Root Object

Field	Type	Description
limit	integer	Number of records returned
offset	integer	Starting index
total	integer	Total number of users matching filters
data	array	List of user records

Data Object

Field	Type	Description
userId	string	Unique user identifier
name	string	Full name of the user
email	string	User email address
phone	string	User phone number
mfaEnabled	boolean	Whether MFA is enabled
lastLogin	string (ISO 8601)	Last login timestamp
status	string	User status (`active`, `inactive`, `suspended`)

Error Responses

400 Bad Request

Invalid filters or query parameters

```
{
 "error": "BadRequest",
 "message": "Invalid query parameters"
}
```

---

401 Unauthorized

```
{
 "error": "Unauthorized",
 "message": "Invalid or missing authentication token"
}
```

---

403 Forbidden

```
{
 "error": "Forbidden",
 "message": "You do not have access to this resource"
}
```

---

500 Internal Server Error

```
{
 "error": "InternalServerError",
 "message": "Failed to fetch users"
}
```

Design Notes (This is where things can go wrong if not careful)

1. Search Implementation (Important)

You're effectively doing:

```
WHERE name ILIKE '%search%'
  OR email ILIKE '%search%'
  OR phone ILIKE '%search%'
```

This becomes slow at scale → later replace with:

- full-text index
- or Elasticsearch

---

2. Filtering Logic

Combine filters safely:

```
WHERE status = ?
AND last_login BETWEEN ? AND ?
AND mfa_enabled = ?
```

---

3. Indexing Strategy (Critical for performance)

You'll want indexes on:

- `status`
- `last_login`
- `email`
- maybe composite `(status, last_login)`

---

4. Offset vs Cursor (Important Insight)

- Offset → simple but slow at high scale
- Cursor → better for infinite scroll

Future upgrade:

GET api.z-tas.com/api/v1/admin/users?cursor=eyJsYXN0TG9naW4iOiIyMDI2...

---

5. Frontend Flexibility

This structure allows:

- table view
- infinite scroll

- filters sidebar
- search-as-you-type

---

6. Real-world Edge Cases

- Users with **no lastLogin** → return `null`
- Phone may be optional
- Name might be empty (handle gracefully)

User Management - Lock / Unlock User

Endpoint

PATCH api.z-tas.com/api/v1/admin/users/{userId}/lock-status

Description

Allows an admin to **lock or unlock a user account** by updating the user's lock status.

- `locked = true` → User is locked (cannot access system)
- `locked = false` → User is unlocked

Request

Path Parameters

```
{
 "userId": "string (required)"
}
```

Request Body

```
{
 "locked": "boolean (required)"
}
```

Example Request

PATCH api.z-tas.com/api/v1/admin/users/12345/lock-status

```
{
 "locked": true
}
```

Response

✅ Success — 200 OK

```
{
 "message": "User lock status updated successfully",
 "data": {
   "userId": "12345",
   "locked": true
 }
}
```

❌ Error Responses

400 Bad Request

Invalid input (e.g., missing or wrong data type)

{
 "error": "Invalid request body",
 "details": [
   {
     "field": "locked",
     "message": "This field is required and must be a boolean"
   }
 ]
}

---

401 Unauthorized

Admin authentication missing or invalid

{
 "error": "Unauthorized",
 "message": "Authentication required"
}

---

403 Forbidden

User is authenticated but not an admin

{
 "error": "Forbidden",
 "message": "Admin privileges required"
}

---

404 Not Found

User does not exist

```
{
 "error": "User not found",
 "message": "No user exists with the given ID"
}
```

---

500 Internal Server Error

Unexpected server failure

```
{
 "error": "Internal server error",
 "message": "Something went wrong"
}
```

Notes

- This endpoint should be **protected** (admin-only).
- Consider auditing/logging this action (who locked/unlocked whom).
- Idempotent behavior: sending the same `locked` value repeatedly should not cause issues.

Utility API — Ping (Client Latency Measurement)

Endpoint

GET api.z-tas.com/api/v1/ping

Description

Lightweight endpoint used by clients to measure **HTTP round-trip time (latency)**.

- The server responds immediately with a timestamp.
- The **client calculates latency** by measuring the time before request and after response.

Request

Headers (Optional but Recommended)

```
{
 "X-Client-Timestamp": "integer (milliseconds since epoch)"
}
```

- Sent by client before making request
- Helps compute latency more precisely (optional)

Response

Success — 200 OK

```
{
 "message": "pong",
 "data": {
   "server_timestamp": 1710000000000
 }
}
```

Field Breakdown

- `server_timestamp` → server time in milliseconds (Unix epoch)

How Latency is Calculated (Client-Side)

Method 1 — Simple (Recommended)

latency = response_received_time - request_sent_time

Method 2 — With Server Timestamp (More Precise)

latency ≈ (response_received_time - request_sent_time) / 2

Assumes symmetric network delay (not always perfect, but closer)

Error Responses

429 Too Many Requests

```
{
 "error": "Too many requests",
 "message": "Rate limit exceeded"
}
```

---

500 Internal Server Error

```
{
 "error": "Internal server error",
 "message": "Ping failed"
}
```

Notes (Important)

- This endpoint should be:
  - **Extremely lightweight** (no DB calls, no logic)
  - **Highly available**
- Do NOT:
  - Add authentication (unless necessary)
  - Add heavy middleware
- Keep response minimal to avoid skewing latency

Device Management — List Devices

Endpoint

GET api.z-tas.com/api/v1/admin//devices

Description

Retrieves a list of devices with support for:

- Pagination (**limit + offset**)
- Search (by device name)
- Filtering (by location, activity, etc.)

Designed to support:

- Infinite scroll
- Traditional page-based pagination

Query Parameters

{

"limit": "integer (optional, default: 20, max: 100)",

"offset": "integer (optional, default: 0)",

"search": "string (optional)",

"location": "string (optional)",

"last_active_from": "integer (optional, timestamp)",

"last_active_to": "integer (optional, timestamp)",

"sort_by": "string (optional, default: last_active)",

"sort_order": "string (optional, asc | desc, default: desc)"

}

Example Request

GET
api.z-tas.com/api/v1/admin/devices?limit=10&offset=20&search=sensor&location=warehouse&sort_by=last_active&sort_order=desc

---

Response

Success — 200 OK

{

"message": "Devices retrieved successfully",

"data": {

"devices": [

{

"device_id": "dev_001",

"device_name": "Temperature Sensor A",

"location": "Warehouse",

"last_active": 1710000000000

},

{

```
      "device_id": "dev_002",
      "device_name": "Humidity Sensor B",
      "location": "Warehouse",
      "last_active": 1709999900000
    }
  ],
  "pagination": {
    "limit": 10,
    "offset": 20,
    "returned": 2,
    "has_more": true
  }
}
}
```

Field Breakdown

`devices[]`

- `device_id` → unique identifier
- `device_name` → human-readable name
- `location` → device location
- `last_active` → last activity timestamp (Unix ms)

`pagination`

- `limit` → requested limit
- `offset` → starting index
- `returned` → number of items returned

- `has_more` → whether more data exists (important for infinite scroll)

Error Responses

400 Bad Request

Invalid query parameters


```
{
 "error": "Invalid query parameters",
 "details": [
   {
     "field": "limit",
     "message": "Must be between 1 and 100"
   }
 ]
}
```


---

401 Unauthorized


```
{
 "error": "Unauthorized",
 "message": "Authentication required"
}
```


---

500 Internal Server Error


```
{
```

```
 "error": "Internal server error",

 "message": "Failed to retrieve devices"

}
```

Design Notes

1. Why `limit + offset`?

- Works for both:
  - Infinite scroll → increment offset
  - Pagination → `offset = page * limit`

---

2. Why NOT page number directly?

Because:

offset = (page - 1) * limit

Page is just abstraction — offset is the real control.

---

3. Search Implementation

- Typically:

```
WHERE device_name ILIKE '%search%'
```

---

4. Filtering Strategy

Each filter should be optional:

```
WHERE

 ($1 IS NULL OR location = $1)

 AND ($2 IS NULL OR last_active >= $2)
```

---

5. Sorting (Critical for Pagination Consistency)

Always enforce deterministic sorting:

ORDER BY last_active DESC, device_id ASC

---

6. Offset Problem (Advanced Insight)

Offset pagination becomes slow at large offsets.

Better alternative (future upgrade):

- Cursor-based pagination (`last_active + id cursor`)
- But offset is fine for now

---

7. Indexing (You WILL need this)

If this grows:

- Index on:
  - `device_name` (for search)
  - `location`
  - `last_active`

Device Management — Force Logout

Endpoint

POST api.z-tas.com/api/v1/admin/devices/{deviceId}/force-logout

Description

Allows an admin to **forcefully terminate a device session**.

- Invalidates the device's active session/token
- The device will be required to re-authenticate

Request

Path Parameters

```
{
 "deviceId": "string (required)"
}
```

Request Body

```
{}
```

No body required — the action is fully defined by the `deviceId`

Example Request

POST api.z-tas.com/api/v1/admin/devices/dev_001/force-logout

Response

✅ Success — 200 OK

```
{
 "message": "Device logged out successfully",
 "data": {
   "device_id": "dev_001",
   "logged_out": true
 }
}
```

Error Responses

400 Bad Request

Invalid device ID format

{

"error": "Invalid device ID",

"message": "The provided device ID is not valid"

}

---

401 Unauthorized

{

"error": "Unauthorized",

"message": "Authentication required"

}

---

403 Forbidden

User is not an admin

{

"error": "Forbidden",

"message": "Admin privileges required"

}

---

404 Not Found

Device does not exist

```
{
 "error": "Device not found",
 "message": "No device exists with the given ID"
}
```

---

409 Conflict (Important Edge Case)

Device already logged out / no active session

```
{
 "error": "Device already logged out",
 "message": "No active session found for this device"
}
```

---

500 Internal Server Error

```
{
 "error": "Internal server error",
 "message": "Failed to logout device"
}
```

Design Notes (Important)

1. What “force logout” actually means

Under the hood, this usually involves:

- Deleting session from DB / cache (e.g., Redis)
- Revoking JWT (via blacklist or versioning)

- Marking device session as invalid

---

2. Why POST (not DELETE or PATCH)?

- This is an **action**, not a resource update
- You're triggering behavior, not modifying state directly

---

3. Idempotency Consideration

You have two valid approaches:

Option A (Strict)

- Return `409` if already logged out (current design)

Option B (Idempotent)

- Always return success:

```
{
 "message": "Device logged out successfully",
 "data": {
   "device_id": "dev_001",
   "logged_out": false
 }
}
```

For admin tools, **idempotent is usually better**

---

4. Security (Critical)

- Must be **admin-protected**
- Should be **audited/logged**:
    - Who logged out which device
    - When

Enforce Multi-Factor Authentication (MFA)

Endpoint

PATCH api.z-tas.com/api/v1/admin/security/mfa-enforcement

Description

Allows admins to **globally enforce or disable two-factor authentication (MFA)** for all users.

- `enabled = true` → All users are required to use MFA
- `enabled = false` → mFA is optional

This typically updates a **system configuration flag**, not individual users directly.

Request

Request Body

{

"enabled": "boolean (required)"

}

Example Request

PATCH api.z-tas.com/api/v1/admin/security/2fa-enforcement

{

"enabled": true

}

Response

Success — 200 OK


```
{
 "message": "Two-factor authentication enforcement updated successfully",
 "data": {
   "enabled": true
 }
}
```

Error Responses

400 Bad Request

Invalid request body


```
{
 "error": "Invalid request body",
 "details": [
   {
     "field": "enabled",
     "message": "This field is required and must be a boolean"
   }
 ]
}
```


---

401 Unauthorized

```
{
 "error": "Unauthorized",
 "message": "Authentication required"
}
```

---

403 Forbidden

Non-admin attempting access

```
{
 "error": "Forbidden",
 "message": "Admin privileges required"
}
```

---

409 Conflict (Optional but Realistic)

If system is already in the requested state

```
{
 "error": "No change required",
 "message": "MFA enforcement is already enabled"
}
```

(You can skip this if you prefer idempotent success — usually better)

---

500 Internal Server Error

```
{
 "error": "Internal server error",
 "message": "Failed to update 2FA enforcement setting"
```

}

Design Notes (Important)

1. What this actually does (under the hood)

This flag should:

- Force mFA setup on next login if not configured
- Block access until 2FA is enabled
- Be checked during authentication flow

---

2. Where to store this?

- Config table in DB (recommended)
- Or cached config (Redis / in-memory)

Example:

key: require_mfa = true

---

3. Idempotency (Recommended Approach)

Better to always return success:

```
{
 "message": "Multi-factor authentication enforcement updated successfully",
 "data": {
   "enabled": true
 }
}
```

Even if no change — avoids unnecessary frontend branching.

---

4. Security Implication (Critical)

- This is a **high-impact action**
- Should be:
  - Logged (audit trail)
  - Possibly require **re-authentication / MFA confirmation for admin**

Authentication Logs

Endpoint

GET api.z-tas.com/api/v1/admin/auth/logs

Description

Retrieves authentication logs with support for:

- Pagination (**limit + offset**) → infinite scroll friendly
- Search (by username / user ID)
- Filtering:
  - Status (`success` / `failed`)
  - Time range (`from` → `to`)

Query Parameters

{

"limit": "integer (optional, default: 20, max: 100)",

"offset": "integer (optional, default: 0)",

"search": "string (optional)",

"status": "string (optional, success | failed)",

"from": "integer (optional, timestamp in ms)",

"to": "integer (optional, timestamp in ms)",

"sort_order": "string (optional, asc | desc, default: desc)"

}

Example Request

GET
api.z-tas.com/api/v1/admin/auth/logs?limit=10&offset=0&search=john&status=failed&from=1709000000000&to=1710000000000

Response

Success — 200 OK

{

"message": "Authentication logs retrieved successfully",

"data": {

"logs": [

{

"log_id": "log_001",

"timestamp": 1710000000000,

"user_name": "john_doe",

"user_id": "user_123",

"status": "failed",

"location": "Singapore",

"device": "Chrome on Windows"

},

{

"log_id": "log_002",

```
      "timestamp": 1709999900000,
      "user_name": "jane_doe",
      "user_id": "user_456",
      "status": "success",
      "location": "Malaysia",
      "device": "iPhone Safari"
    }
  ],
  "pagination": {
    "limit": 10,
    "offset": 0,
    "returned": 2,
    "has_more": true
  }
}
}
```

Field Breakdown

`logs[]`

- `log_id` → unique log identifier
- `timestamp` → event time (Unix ms)
- `user_name` → username at time of login
- `user_id` → unique user ID
- `status` → `success` or `failed`
- `location` → derived from IP (approximate)
- `device` → user agent / parsed device info

Error Responses

400 Bad Request

Invalid query parameters


```
{
 "error": "Invalid query parameters",
 "details": [
   {
     "field": "status",
     "message": "Must be either 'success' or 'failed'"
   }
 ]
}
```


---

401 Unauthorized


```
{
 "error": "Unauthorized",
 "message": "Authentication required"
}
```


---

403 Forbidden


```
{
 "error": "Forbidden",
```

```
 "message": "Admin privileges required"
}
```

---

500 Internal Server Error

```
{
 "error": "Internal server error",
 "message": "Failed to retrieve authentication logs"
}
```

Design Notes (Important)

1. Search Strategy

Search should match:

```
WHERE user_name ILIKE '%search%'
  OR user_id ILIKE '%search%'
```

---

2. Filtering Logic

```
WHERE
 ($1 IS NULL OR status = $1)
 AND ($2 IS NULL OR timestamp >= $2)
 AND ($3 IS NULL OR timestamp <= $3)
```

---

3. Sorting (Critical)

Always enforce deterministic order:

ORDER BY timestamp DESC, log_id DESC

---

4. Pagination

Same reasoning as before:

offset = page * limit

---

5. Indexing (You WILL need this at scale)

- `timestamp` (for sorting + range queries)
- `status`
- `user_id`
- Optional: full-text index for `user_name`

---

6. Data Sensitivity (Important)

Auth logs are **security-critical**:

- Must be **admin-only**
- Should be **immutable**
- Consider retention policy (e.g., 30–90 days)

---

7. Real-World Upgrade (When scale hits)

Offset pagination will degrade → move to:

- Cursor-based pagination (`timestamp + log_id` cursor)

Authentication Analytics

Endpoint

GET api.z-tas.com/api/v1/admin/auth/analytics

Description

Provides aggregated authentication statistics for the **last 24 hours**:

- Number of successful logins
- Number of failed login attempts
- Number of suspicious activities

Query Parameters (Optional — Future-Proofing)


```
{
 "from": "integer (optional, timestamp in ms)",
 "to": "integer (optional, timestamp in ms)"
}
```


If not provided, defaults to **last 24 hours**

Example Request

GET /admin/auth/analytics

Response

Success — 200 OK


```
{
 "message": "Authentication analytics retrieved successfully",
 "data": {
   "time_range": {
     "from": 1709913600000,
```

```
    "to": 1710000000000
  },
  "metrics": {
    "successful_logins": 1245,
    "failed_logins": 312,
    "suspicious_activities": 27
  }
 }
}
```

Field Breakdown

`time_range`

- `from` → start timestamp
- `to` → end timestamp

`metrics`

- `successful_logins` → count where `status = success`
- `failed_logins` → count where `status = failed`
- `suspicious_activities` → flagged events (see below)

Error Responses

400 Bad Request

Invalid time range


```
{
 "error": "Invalid query parameters",
```

"details": [
  {
    "field": "from/to",
    "message": "Invalid or inconsistent time range"
  }
]
}

---

401 Unauthorized

{
 "error": "Unauthorized",
 "message": "Authentication required"
}

---

403 Forbidden

{
 "error": "Forbidden",
 "message": "Admin privileges required"
}

---

500 Internal Server Error

{
 "error": "Internal server error",
 "message": "Failed to retrieve analytics"

}

Design Notes (This is where the real thinking is)

1. What counts as “suspicious”?

You **must define this explicitly**. Common strategies:

- Multiple failed attempts within short time window
- Login from unusual location/device
- Rapid IP switching
- Known malicious IPs

Example (simple rule):

failed_attempts >= 5 within 10 minutes → suspicious

---

2. Query Strategy (Efficient Aggregation)

Instead of 3 separate queries, use conditional aggregation:

```
SELECT

 COUNT(*) FILTER (WHERE status = 'success') AS successful_logins,

 COUNT(*) FILTER (WHERE status = 'failed') AS failed_logins,

 COUNT(*) FILTER (WHERE is_suspicious = true) AS suspicious_activities

FROM auth_logs

WHERE timestamp BETWEEN $1 AND $2;
```

---

3. Performance Considerations

If logs grow large:

- Use **indexes on `timestamp`, `status`, `is_suspicious`**
- Consider **pre-aggregated tables / caching**

---

4. Why this endpoint matters

This is what powers:

- Admin dashboards
- Security monitoring panels
- Alerts systems

---

5. Future Upgrade (Real Dashboard Needs)

You'll eventually want:

- Time-series data (per hour breakdown)
- Graph-ready format:

```
{

 "timestamps": [...],

 "success": [...],

 "failed": [...]

}
```

Key Insight

This endpoint is fundamentally different from logs:

- Logs → raw data (GET /logs)
- Analytics → computed summary (GET /analytics)

Mixing them is a common design mistake — you avoided it